
VISION-PREDICTOR-ACTION: TOWARDS EFFICIENT EMBODIED AI VIA NEURO-SYMBOLIC JEPA PREDICTORS

Yeonseok Lee
SLING AI Inc.
Incheon, Republic of Korea
ylee@sling.ai.kr

ABSTRACT

Vision-Language-Action (VLA) models for embodied manipulation commonly rely on autoregressive Large Language Model (LLM) decoders, whose number of sequential forward passes grows with the length of the tokenized action sequence and thereby limits the achievable closed-loop control frequency. To address this bottleneck, we propose the Goal-Conditioned Vision-Predictor-Action (VPA) framework, which replaces the autoregressive LLM backbone with a Joint-Embedding Predictive Architecture (JEPA) predictor and decouples high-level task intent from low-level continuous motor generation. The pipeline integrates a lightweight frozen text encoder for natural language instructions, a shared vision encoder that embeds observations and milestone goals into a compact latent space, a non-autoregressive neuro-symbolic primitive selector, a JEPA predictor that forecasts state transitions directly as continuous latent vectors, and a conditional flow-matching solver that generates multi-step action chunks. Because the solver produces an entire H -step action chunk jointly through K numerical ODE integration steps (typically $K \in \{1, 2, 3\}$), the number of sequential solver evaluations per control cycle is $\mathcal{O}(K)$ and independent of H , in contrast to the $\Theta(H \cdot d_a)$ sequential decoding steps of token-based action generation; the arithmetic cost of each evaluation, however, still scales with the chunk dimensionality $H \cdot d_a$. Discrete primitive symbols are confined to the high-level selector, whereas motor commands are conditioned on continuous predicted latents; the repertoire of high-level behaviors therefore remains bounded by the primitive dictionary. By avoiding pixel-level reconstruction and text-based autoregression, the framework models environment dynamics within a compact latent space. We complement the architecture with an analysis of its inference complexity, its anti-collapse latent regularization, and the regularity assumptions on the vision encoder under which latent sub-goal thresholding can be expected to limit error accumulation across task phases. The resulting design targets a low-latency, high-frequency latent control loop for reactive embodied manipulation, and we outline an evaluation protocol for its empirical validation.

Keywords Embodied AI · Neuro-Symbolic AI · Joint-Embedding Predictive Architecture · Flow Matching · Robotic Manipulation

1 Introduction

Recent advances in embodied artificial intelligence have been largely driven by Vision-Language-Action (VLA) models, which extend web-scale pre-trained backbones to physical robotic manipulation [1, 2]. By mapping visual observations and natural language instructions into discrete token spaces, these systems exhibit strong semantic generalization across diverse tasks [3]. However, many current VLA architectures emit actions through autoregressive Large Language Model (LLM) decoders [4]. When continuous actions are discretized into tokens, generating an H -step action chunk of dimension d_a requires $N = q \cdot H \cdot d_a$ tokens, where q denotes the number of tokens per scalar action component, and hence $\Theta(N)$ sequential forward passes through a multi-billion-parameter Transformer per execution cycle [4]. This sequential dependency limits closed-loop control frequencies to approximately 2–5 Hz (200–500 ms latency) [4], which restricts the ability of autoregressive VLAs to execute reactive, dynamic maneuvers or to compensate for unexpected physical perturbations in real time.

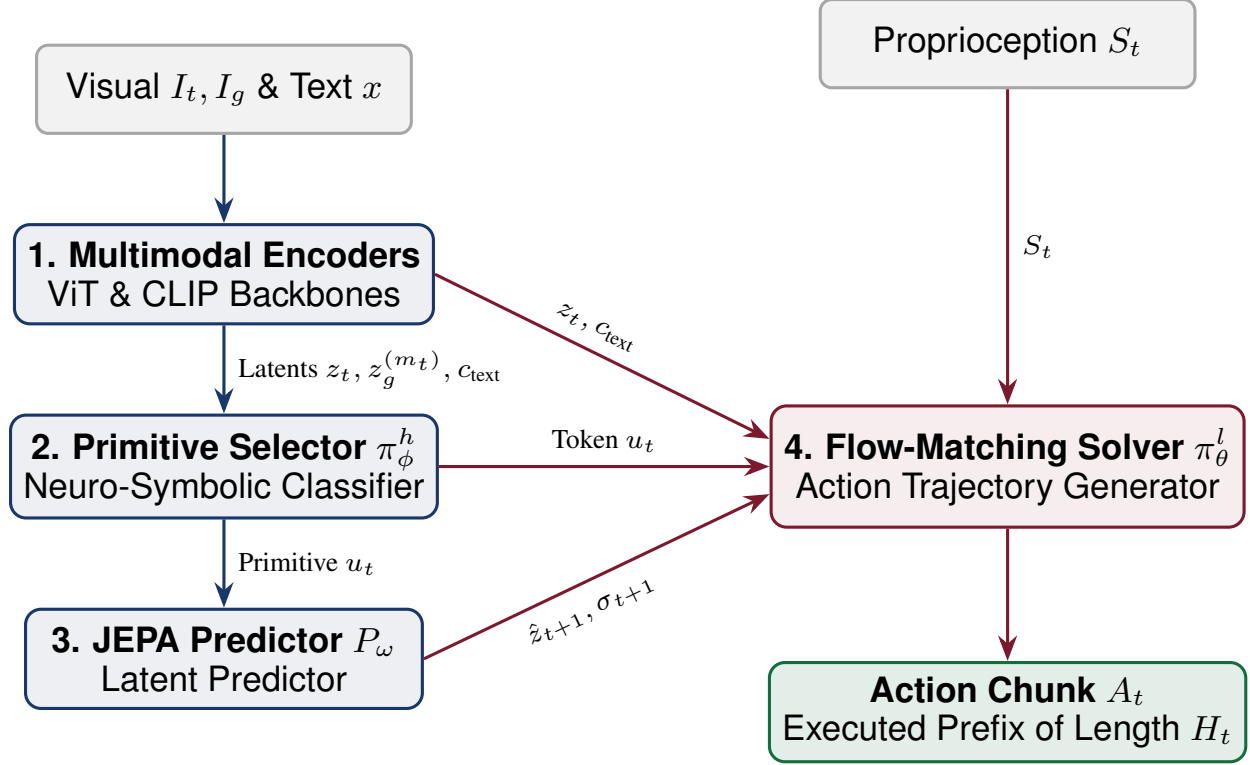


Figure 1: High-level architectural overview of the Goal-Conditioned Vision-Predictor-Action (VPA) framework. Multimodal observations are projected into latent and text embeddings that condition the selection of an active primitive u_t , the forecast of the next latent state $\hat{z}_{t+1}$ and its uncertainty σ_{t+1} by a JEPA predictor, and a continuous flow-matching solver that generates an action chunk A_t , of which an uncertainty-dependent prefix of length H_t is executed.

To mitigate these latency constraints, hybrid approaches such as NS-VLA [4] introduce discrete neuro-symbolic skill pointers, while VLA-JEPA [5] incorporates joint-embedding predictive architectures (JEPA) [6, 7] to model latent world dynamics [8, 9]. Despite these improvements, such systems remain coupled to large causal vision-language backbones (e.g., Qwen3-VL-2B) that consume substantial computational resources, and their high-level plans are expressed as predefined action symbols (e.g., *pick*, *place_on*) that constitute the interface between reasoning and control. Concurrently, non-autoregressive trajectory generators based on Action Chunking with Transformers (ACT) [10], Diffusion Policies [11], and Conditional Flow-Matching (CFM) [12, 13, 14] have demonstrated high-frequency motor execution (30–100 Hz). These methods, however, typically either operate as reactive visual policies without forward-predictive dynamics or explicit sub-goal tracking, or, when language-conditioned as in π_0 [14], retain a large vision-language backbone for instruction processing.

In this work, we present the **Goal-Conditioned Vision-Predictor-Action (VPA)** framework, which replaces autoregressive language decoding with a non-autoregressive latent control loop. By substituting the LLM decoder—the dominant source of sequential latency in token-based VLA models—with a lightweight JEPA predictor, VPA is designed to reduce both parameter count and inference latency. Rather than communicating through text tokens, the predictor supplies forecasted state-transition latents directly to the downstream action solver, which generates continuous and potentially multi-modal action distributions within the skill selected by a discrete primitive. VPA thus retains a symbolic vocabulary for high-level skill selection, but confines it to the choice of *which* skill is executed, while the motor trajectory itself is synthesized in a continuous space. We note that this design does not remove the symbolic bottleneck entirely: the set of high-level behaviors that VPA can compose is bounded by its primitive dictionary, and zero-shot generalization to skills outside this dictionary is not addressed by the present formulation. As illustrated in Figure 1, VPA decouples high-level task logic from low-level continuous motor control across a four-stage pipeline:

1. **Latent Multimodal Goal Encoding (E_ψ and Text Encoder):** A non-generative, self-supervised vision encoder [15, 16] maps sensory frames into a compact latent space $\mathcal{Z} \subset \mathbb{R}^d$, in which Euclidean distances $\|z_t - z_g\|_2$ are intended to serve as proxies for physical task progress [4]. Concurrently, a lightweight, frozen

text encoder (e.g., CLIP [17]) embeds the user’s natural language instruction x into a dense vector c_{text} , injecting the overarching task intent without requiring an LLM decoder.

2. **Neuro-Symbolic Primitive Selector (π_ϕ^h):** A non-autoregressive classifier selects the active primitive token $u_t \in \mathcal{U}$ from a fixed dictionary of N_u atomic skills, conditioned on the current latent z_t , the milestone latent $z_g^{(m_t)}$, and the text embedding c_{text} . This multimodal conditioning allows the selector to disambiguate task intent (e.g., “move the cup” versus “throw it away”). Milestone progression is governed by a symbolic transition rule [18] based on the latent threshold test $\|z_t - z_g^{(m_t)}\|_2 < \tau$ [4], where τ is scaled by the running latent variance margin maintained by the VICReg objective rather than fixed a priori (Section 4.2).
3. **JEPA Predictor (P_ω):** A lightweight predictive dynamics model forecasts the next latent state together with a scalar uncertainty estimate, $(\hat{z}_{t+1}, \sigma_{t+1}) = P_\omega(z_t, u_t)$, and is regularized against representation collapse via Variance-Invariance-Covariance Regularization (VICReg) [19, 8]. The predictor is trained with conditioning on discrete primitive tokens and subsequently frozen, so that the downstream solver is trained against a stationary conditioning distribution rather than against latent inputs that drift as the representation evolves.
4. **Flow-Matching Action Solver (π_θ^l):** A conditional vector field v_θ synthesizes a continuous action chunk $A_t \in \mathbb{R}^{H \times d_a}$ jointly over all H steps using $K \in \{1, 2, 3\}$ numerical ODE integration steps [12], of which a prefix of length $H_t \leq H$, adapted to the predictor’s uncertainty σ_{t+1} , is executed.

Summary of Theoretical Contributions:

- **Horizon-Independent Sequential Depth:** We show that replacing autoregressive token generation with flow-matching action chunking reduces the number of sequential network evaluations per control cycle from $\Theta(N)$, with $N = q \cdot H \cdot d_a$, to $K + 3 = \mathcal{O}(K)$, a quantity independent of the action horizon H . The arithmetic cost of each solver evaluation nevertheless grows with the chunk dimensionality $H \cdot d_a$; wall-clock latency is therefore approximately horizon-independent only while this computation fits within the available hardware parallelism. We accordingly state the ≤ 10 ms inference latency and 30–100 Hz control frequency as design targets subject to empirical validation, rather than as guarantees.
- **Anti-Collapse Representation Analysis:** We characterize the conditions under which the variance hinge penalty $v(Z)$ and the covariance decorrelation penalty $c(Z)$ exclude the constant (fully collapsed) representation from the global minimizers of the joint-embedding objective, without relying on pixel-reconstruction decoders. We further use the realized variance margin to calibrate the latent completion threshold τ , so that τ remains commensurate with the scale of the latent space throughout training.
- **Conditional Error Containment Across Task Phases:** We make explicit the encoder regularity conditions under which discrete sub-goal thresholding $\|z_t - z_g^{(m_t)}\|_2 < \tau$ limits the physical deviation at each phase transition. These are an upper Lipschitz constant L_E , which can be enforced with respect to the input image via spectral normalization [20] and governs whether the transition is triggered as the physical state approaches the milestone, and a lower (co-Lipschitz) constant μ_E with respect to task-relevant physical state, which bounds the physical deviation at transition by τ/μ_E . Because standard Vision Transformers are not globally Lipschitz [21], and because the lower constant depends on the relation between images and physical state and cannot be enforced through architectural constraints alone, we present this error containment as an empirical expectation to be verified rather than as a formal guarantee.

2 Related Work

2.1 Vision-Language-Action Models

Recent advancements in embodied artificial intelligence have been largely driven by Vision-Language-Action (VLA) models, such as RT-2 [1], OpenVLA [2], and Instruct2Act [3]. These architectures extend web-scale pre-trained backbones to map visual observations and natural language instructions into multi-modal action spaces. However, VLAs that emit discretized action tokens through an autoregressive decoder incur an inherent sequential bottleneck: generating a multi-step action chunk requires N sequential forward passes through a multi-billion-parameter Transformer decoder, where N grows linearly with the action horizon, resulting in $\Theta(N)$ sequential decoding steps per execution cycle. This overhead has been reported to limit closed-loop control frequencies to approximately 2–5 Hz [4], reducing the effectiveness of autoregressive VLAs for real-time dynamic manipulation and for compensating physical perturbations. Generalist policies that replace token decoding with continuous generative action heads, such as Octo [22] and π_0 [14], partially alleviate this bottleneck; we discuss these approaches in Section 2.4.

2.2 Neuro-Symbolic Robotics

To combine high-level semantic reasoning with continuous motor execution, neuro-symbolic robotics incorporates formal logic and discrete task structures. Classical approaches utilize Linear Temporal Logic (LTL) and automata to enforce reactive mission planning and safety constraints [18]. More recently, hybrid frameworks like NS-VLA [4] introduce discrete skill pointers to decompose complex manipulation tasks into atomic primitives. Nevertheless, existing neuro-symbolic VLA architectures continue to rely on causal Vision-Language Models (VLMs) or autoregressive text parsers for instruction processing and high-level logic; consequently, they inherit the sequential inference latency bottleneck, which limits high-frequency closed-loop execution. Concurrent work has also begun to examine neuro-symbolic formulations of JEPA, including approaches that couple knowledge and rule embeddings with joint-embedding prediction [23] and hybrid symbolic-neural latent predictors [24]. Our framework builds upon the principles of Neuro-Symbolic Joint-Embedding Predictive Architecture (NS-JEPA) [25], replacing autoregressive language parsers with a non-autoregressive primitive selector coupled directly to a latent predictive world model. We emphasize that the symbolic component of VPA is deliberately minimal: its transition rule (Section 4.2) realizes a monotone sequential automaton over a fixed primitive dictionary rather than a general temporal-logic specification, and it therefore inherits the expressivity limits of that dictionary.

2.3 Predictive World Models and Joint-Embedding Architectures

In contrast to generative world models that reconstruct high-dimensional visual pixels, Joint-Embedding Predictive Architectures (JEPAs) [6, 7] forecast future environment dynamics directly within abstract latent spaces. Because they dispense with pixel-level reconstruction decoders, JEPAs are not required to allocate model capacity to stochastic visual nuisances such as dynamic lighting and background shifts. Recent works, including LeWorldModel [8] and hierarchical latent planning frameworks [9], construct stable, non-collapsing latent spaces for world modeling and planning. The regularizers employed in this line of work range from variance–covariance penalties such as VICReg [19] to distribution-matching objectives that encourage isotropic Gaussian embeddings, as in LeJEPA [26]. At scale, V-JEPA 2 [27] shows that an action-conditioned latent predictor, post-trained on a modest amount of robot interaction data, can support zero-shot manipulation through planning in representation space, albeit at the cost of iterative optimization over action sequences at inference time.

In robotic control, joint-embedding mechanics have been applied both as predictive dynamics models and as visual feature extractors. VLA-JEPA [5] introduces a unified pretraining framework with an online, time-causal latent world model to forecast future states. Conversely, JEPA-VLA [28] argues that standard visual backbones lack temporal policy priors and employs frozen video-predictive JEPA features as plug-and-play visual representations inside existing VLA architectures via gated cross-attention. Additionally, ACT-JEPA [29] employs latent prediction to regularize continuous imitation learning policies. While these approaches enhance visual representations or latent forward modeling, they either lack explicit, deterministic state-machine mechanisms for sub-goal verification or remain coupled to latency-heavy, text-conditioned causal language backbones.

2.4 Diffusion and Flow-Matching Policies

To enable high-frequency continuous control, non-autoregressive trajectory generation methods have emerged as an effective alternative. Action Chunking with Transformers (ACT) [10] predicts multi-step continuous action trajectories $A_t \in \mathbb{R}^{H \times d_a}$ in a single forward pass, which reduces the effective number of decisions per episode and has been shown to mitigate compounding errors in imitation learning. Building upon this, Diffusion Policies [11] and Conditional Flow Matching (CFM) [12, 13] formulate action-chunk generation as iterative denoising or as transport along a learned ODE vector field, respectively, enabling expressive multi-modal trajectory sampling; straight or nearly straight transport paths further permit accurate integration in few steps [30]. When combined with action chunking, these continuous solvers support real-time execution rates (30–100 Hz). However, they typically function either as reactive visual policies that lack long-horizon predictive dynamics, symbolic verification, or explicit sub-goal state tracking, or as action heads attached to generalist transformer backbones [22], which in the case of π_0 [14] is a pre-trained vision-language model with billions of parameters. VPA adopts a CFM solver for low-level generation but conditions it on forecasted latent transitions and a lightweight text embedding, rather than on the hidden states of a large language backbone.

3 Problem Formulation

We formulate visual-goal robotic manipulation as a Goal-Conditioned Markov Decision Process (G-MDP) defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{G}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma_{\text{disc}})$, where $\mathcal{S}$ represents the continuous state space, $\mathcal{G}$ denotes the multimodal goal space, $\mathcal{A}$ is the continuous action-chunk space, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ represents the stochastic chunk-level transition

dynamics, $\mathcal{R}$ is the goal-conditioned (milestone-dependent) reward function, and $\gamma_{\text{disc}} \in [0, 1]$ is the discount factor. We write γ_{disc} to distinguish the discount factor from the VICReg variance margin γ introduced in Section 5.2.

3.1 State, Observation, and Goal Spaces

At control step t , the system state $s_t \in \mathcal{S}$ pairs raw visual sensory inputs with low-level kinematic feedback:

$$s_t = (I_t, S_t) \in \mathcal{S} \quad (1)$$

where $I_t \in \mathbb{R}^{H_{\text{img}} \times W_{\text{img}} \times C}$ denotes the raw RGB observation frame with height H_{img} , width W_{img} , and C channels, while $S_t \in \mathbb{R}^{d_s}$ represents the instantaneous robot proprioceptive state (e.g., joint positions, joint velocities, and end-effector pose).

The pair s_t is an observation of an underlying physical configuration that is not directly accessible. We denote by $y_t \in \mathcal{Y} \subset \mathbb{R}^{d_y}$ the task-relevant physical state (e.g., the robot joint configuration and the poses of manipulated objects) and model image formation as $I_t = \mathcal{I}(y_t, \eta_t)$, where $\mathcal{I}$ is the image-formation (rendering) map and η_t collects task-irrelevant nuisance factors such as illumination, background content, and sensor noise. Following common practice, we treat s_t as the agent’s state, implicitly assuming that it is approximately Markovian for the control task; the distinction between y_t and I_t is made explicit only in the analysis of Section 5.3.

To form a Goal-Conditioned VPA framework, the overarching task intent is specified by natural language alongside visual targets. The multimodal goal space $\mathcal{G}$ is defined by a tuple pairing a user language instruction x with target goal frames I_g :

$$g = (x, I_g) \in \mathcal{G} \quad (2)$$

The language string x provides the semantic intent of the task. For multi-stage compositional tasks, the visual component I_g is an ordered sequence of M visual milestone goals $(I_g^{(1)}, I_g^{(2)}, \dots, I_g^{(M)})$, with $I_g^{(m)} \in \mathbb{R}^{H_{\text{img}} \times W_{\text{img}} \times C}$; single-stage tasks correspond to $M = 1$. Each milestone frame is an observation of a target physical configuration, $I_g^{(m)} = \mathcal{I}(y_g^{(m)}, \eta_g^{(m)})$, and its latent embedding is denoted $z_g^{(m)} = E_\psi(I_g^{(m)})$.

3.2 Action Space and Action Chunking

To account for sensorimotor latency and to reduce single-step compounding execution drift, the controller operates over continuous multi-step action chunks rather than instantaneous atomic commands. Let $\mathcal{A}_0 \subseteq \mathbb{R}^{d_a}$ denote the per-step action space, where d_a is the dimensionality of the robot’s action space (e.g., 7-DoF joint command vectors alongside 1-DoF parallel-gripper actuation). At step t , the controller selects an open-loop action chunk over horizon H :

$$A_t = (a_t, a_{t+1}, \dots, a_{t+H-1}) \in \mathcal{A} = \mathcal{A}_0^H \subseteq \mathbb{R}^{H \times d_a} \quad (3)$$

As detailed in Section 4.4, the solver always generates a chunk of length H , of which an uncertainty-dependent prefix of length $H_t \leq H$ is executed before the inference pipeline is invoked again. We refer to the control steps at which the pipeline is invoked as *decision steps*; the primitive u_t , the milestone pointer m_t , and the conditioning vector e_t are evaluated at decision steps and held fixed during chunk execution.

3.3 Transition Dynamics and Latent Metric Reward

The ground-truth physical dynamics of the environment govern state transitions at the chunk level according to:

$$s_{t+H} \sim \mathcal{P}(\cdot \mid s_t, A_t) \quad (4)$$

where $\mathcal{P}$ is induced by composing the per-step dynamics over the actions of the chunk (or over its executed prefix of length H_t).

Rather than predicting high-dimensional pixel transitions $p(I_{t+H} \mid I_t, A_t)$ directly, the dynamics are modeled in a compact latent space $\mathcal{Z} \subset \mathbb{R}^d$ obtained via the visual encoder E_ψ . Within $\mathcal{Z}$, the latent transition dynamics are modeled by the forward predictor P_ω under the active discrete primitive token $u_t \in \mathcal{U}$:

$$z_t = E_\psi(I_t), \quad (\hat{z}_{t+1}, \sigma_{t+1}) = P_\omega(z_t, u_t) \quad (5)$$

where $\sigma_{t+1} \geq 0$ is a scalar predictive uncertainty (Section 4.3). The predictor operates at a fixed temporal stride of $\nu \in \mathbb{N}$ control steps, and we assume $\nu \leq H_{\min}$, where $H_{\min}$ is the minimum executed chunk length (Section 4.4), so that the forecast horizon falls within every executed chunk. For notational brevity, we write z_{t+1} and $\hat{z}_{t+1}$ for the actual and forecast latents $z_{t+\nu}$ and $\hat{z}_{t+\nu}$ throughout. Because P_ω is conditioned on the primitive u_t rather than on the action chunk A_t , it models the expected latent consequence of executing a skill, as represented in the demonstration

data, rather than the consequence of a specific motor trajectory; the motor-level realization is delegated to the solver of Section 4.4.

The sparse goal-reaching objective is approximated by a dense latent proxy reward based on the Euclidean distance to the active milestone:

$$\mathcal{R}(s_t, g; m_t) = - \left\| E_\psi(I_t) - E_\psi(I_g^{(m_t)}) \right\|_2 = - \left\| z_t - z_g^{(m_t)} \right\|_2 \quad (6)$$

where $m_t \in \{1, \dots, M\}$ is the active milestone pointer. Since $\mathcal{R}$ depends on m_t , the pointer acts as a discrete component of an augmented state (s_t, m_t) . In the present formulation, $\mathcal{R}$ serves as an analysis and monitoring signal rather than as an optimization target, as all components of VPA are trained by self-supervised latent prediction or by imitation of demonstrations (Section 4). Whether this proxy is monotonically related to physical task progress depends on the regularity of E_ψ with respect to y_t , which we examine in Section 5.3.

Milestone m_t is regarded as completed when the latent distance falls below a completion threshold $\tau > 0$, such that $\|z_t - z_g^{(m_t)}\|_2 < \tau$. Because the scale of $\mathcal{Z}$ is not fixed by the joint-embedding objective, τ is calibrated relative to the running latent variance margin rather than set to an absolute constant (Section 4.2). The task is complete when this condition is satisfied for the final milestone $m_t = M$.

4 The Goal-Conditioned VPA Architecture

Current Vision-Language-Action (VLA) architectures that decode actions autoregressively are constrained by a sequential computing bottleneck on control frequency. To overcome this, we introduce the Goal-Conditioned Vision-Predictor-Action (VPA) framework. The architecture bypasses large language model (LLM) backbones and formulates robotic manipulation as a closed-loop latent prediction problem.

The architecture decouples high-level task logic from low-level continuous motor control through a four-stage pipeline: (1) a latent multimodal goal encoder that projects raw observations and text into a compact latent space; (2) a non-autoregressive neuro-symbolic selector that discretizes task primitives; (3) a collapse-regularized latent predictor that forecasts state transitions; and (4) a continuous flow-matching solver that outputs multi-step action trajectories. The number of sequential network evaluations per decision step in this pipeline does not depend on the action horizon (Section 5.1), and the structure is designed to target control frequencies of 30–100 Hz, bridging semantic task specification and reactive control. The complete data flow is illustrated in Figure 2.

4.1 Latent Multimodal Goal Encoding

The perception module in the VPA framework is designed to retain task-relevant spatial and physical configurations while suppressing high-frequency visual nuisance factors η_t (e.g., lighting variations, reflective glare, or dynamic backgrounds). Let $I_t \in \mathbb{R}^{H_{\text{img}} \times W_{\text{img}} \times C}$ denote the current observation frame at timestep t , and let $I_g^{(m)} \in \mathbb{R}^{H_{\text{img}} \times W_{\text{img}} \times C}$ denote the visual goal frame of milestone $m \in \{1, \dots, M\}$.

We employ a shared vision encoder E_ψ (e.g., a Vision Transformer backbone [15]), which operates as a Siamese network to independently map the high-dimensional image frames into a compact, continuous latent embedding space $\mathcal{Z} \subset \mathbb{R}^d$:

$$z_t = E_\psi(I_t), \quad z_g^{(m)} = E_\psi(I_g^{(m)}), \quad m = 1, \dots, M \quad (7)$$

where z_t is intended to encode the current configuration of the robot end-effector and scene objects, and $z_g^{(m)}$ the target configuration of milestone m . The milestone embeddings depend only on the task specification and can be computed once per episode.

To capture the overarching task intent without relying on a computationally heavy LLM, we introduce a lightweight, frozen text encoder (e.g., CLIP [17] or a distilled RoBERTa [31]). This encoder embeds the user’s natural language instruction x into a dense text vector $c_{\text{ext}} \in \mathbb{R}^{d_c}$, which is likewise computed once per episode.

Unlike standard autoencoders or generative models that minimize a pixel-level reconstruction loss $\mathcal{L}_{\text{MSE}}(I_t, \hat{I}_t)$, E_ψ is optimized via a joint-embedding predictive objective (Section 4.3). Because this objective does not require the representation to retain information that is unpredictable from one frame to the next, it encourages, but does not guarantee, invariance to the nuisance factors η_t and a focus on predictable physical quantities such as kinematics, spatial geometry, and relative object distances. The latent distance $\|z_t - z_g^{(m)}\|_2$ is accordingly intended to serve as a proxy for physical task progress; the extent to which it does so is governed by the regularity of the composite map $y \mapsto E_\psi(\mathcal{I}(y, \eta))$ from physical state to latent, which we analyze in Section 5.3. In the base formulation, no explicit Lipschitz constraint is imposed on E_ψ ; the consequences of this choice, and the role of optional spectral normalization,

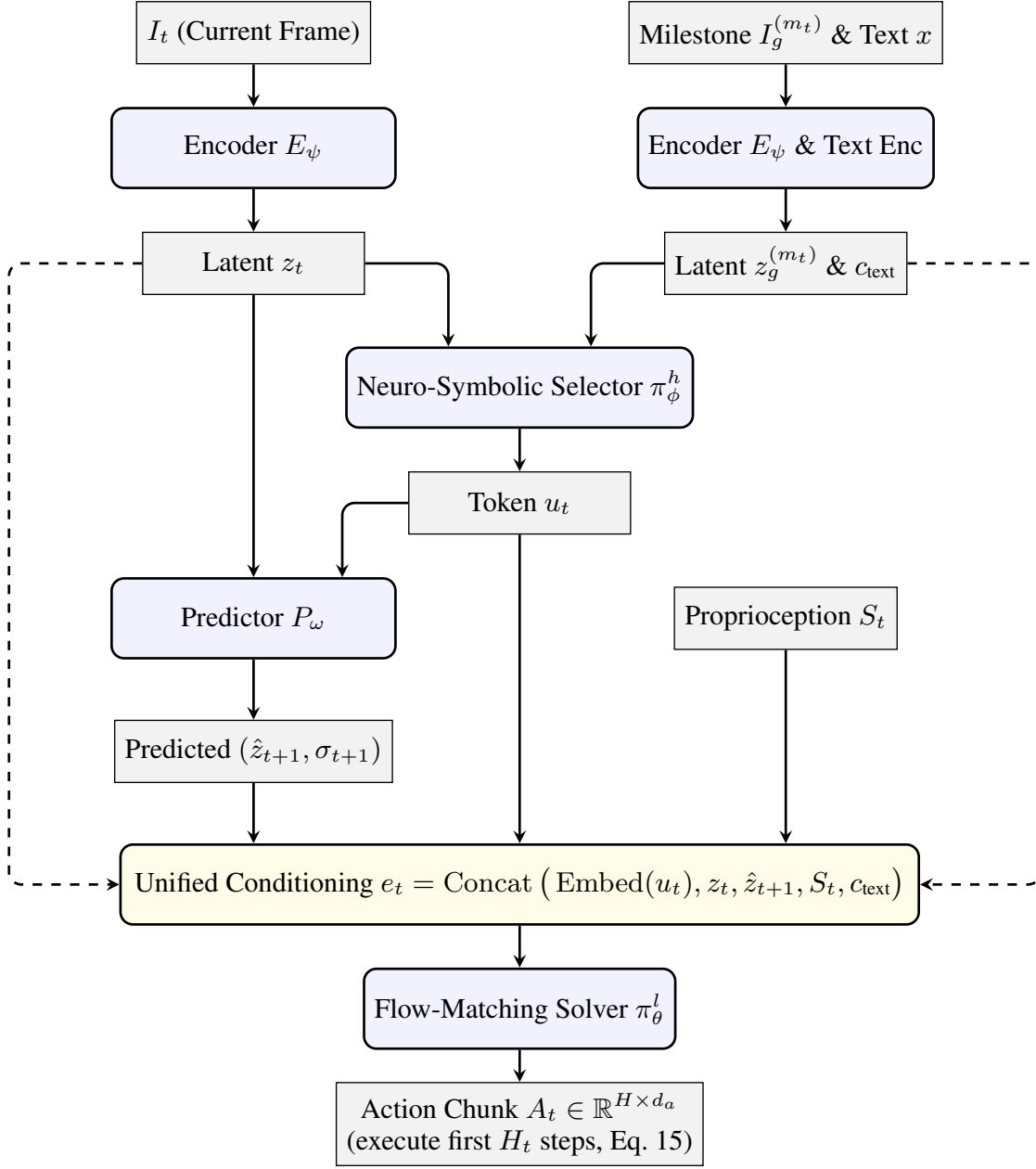


Figure 2: VPA Closed-Loop Inference Pipeline (Action Trajectory Generation). This diagram shows the online forward pass executed at each decision step during robot deployment. The current observation I_t , the active milestone goal $I_g^{(m_t)}$, and the natural language instruction x are encoded and used to select a primitive u_t , forecast $(\hat{z}_{t+1}, \sigma_{t+1})$, and form the unified conditioning vector e_t (Eq. 12), from which the solver synthesizes an action chunk A_t . The predictive uncertainty σ_{t+1} determines the executed prefix length H_t (Eq. 15), and the milestone pointer m_t is updated by the threshold rule of Eq. 9a.

are discussed in the same section. The text embedding c_{text} provides the semantic conditioning for the discrete primitive transitions and the continuous action generation modeled in subsequent stages.

4.2 Neuro-Symbolic Primitive Selector (π_ϕ^h)

A fundamental limitation of autoregressive VLA models is their reliance on computationally heavy text generation to dictate sequential task logic. The VPA architecture instead formulates high-level reasoning as a discrete, latent-conditioned classification problem governed by a symbolic finite-state machine. The per-decision cost of this stage is a single classifier evaluation, independent of the number of milestones M .

Let $\mathcal{U} = \{u^{(1)}, u^{(2)}, \dots, u^{(N_u)}\}$ define a pre-determined dictionary of N_u executable atomic skills, or “primitives” (e.g., *reach*, *grasp*, *push*, *place*). To handle multi-stage tasks, the overarching visual goal is specified as a sequence of milestones, represented in the latent space as $(z_g^{(1)}, z_g^{(2)}, \dots, z_g^{(M)})$ (Eq. 7).

We introduce a discrete, monotonically non-decreasing milestone pointer $m_t \in \{1, 2, \dots, M\}$ that tracks the active sub-goal at decision step t . The neuro-symbolic primitive selector is a lightweight classifier π_ϕ^h (parameterized by a Multi-Layer Perceptron) that maps the current state embedding, the active milestone embedding, and the text embedding c_{text} directly to a categorical primitive token u_t :

$$u_t = \arg \max_{u \in \mathcal{U}} \pi_\phi^h(u \mid z_t, z_g^{(m_t)}, c_{\text{text}}) \quad (8)$$

The selector is trained by minimizing the cross-entropy between π_ϕ^h and the primitive labels provided by the demonstration data (Figure 3). By incorporating c_{text} as a conditioning variable, the selector can disambiguate task intent (e.g., whether the instruction is to “move the cup” or to “throw the cup away”) without generating text tokens. It acts as a reactive policy that selects the skill expected to satisfy both the semantic instruction and the geometric goal by reducing the latent discrepancy between z_t and $z_g^{(m_t)}$.

The progression of the task is governed by a symbolic transition rule that tests sub-goal completion via an ℓ_2 -norm threshold in $\mathcal{Z}$. A threshold τ fixed in absolute units, however, is ill-suited to a jointly trained latent space: the variance hinge of the VICReg objective (Eq. 18) penalizes per-dimension standard deviations below the margin γ but does not bound them from above, so the overall scale of $\mathcal{Z}$ can drift during training, and a static τ would become relatively looser or stricter as training proceeds. We therefore scale τ with a running estimate of the realized latent variance margin. Let $Z^{(n)} \in \mathbb{R}^{B \times d}$ denote the batch of online-encoder latents at training iteration n (Section 5.2). The milestone pointer is updated at each decision step, and the threshold is calibrated during training, according to:

$$m_{t+1} = \begin{cases} m_t + 1, & \text{if } \|z_t - z_g^{(m_t)}\|_2 < \tau \text{ and } m_t < M, \\ m_t, & \text{otherwise,} \end{cases} \quad (9a)$$

$$\bar{\gamma}_n = (1 - \alpha_\gamma) \bar{\gamma}_{n-1} + \alpha_\gamma \sqrt{\frac{1}{d} \sum_{j=1}^d \text{Var}(Z_{:,j}^{(n)})} + \epsilon, \quad (9b)$$

$$\tau_n = \kappa \sqrt{2d} \bar{\gamma}_n, \quad (9c)$$

where $\alpha_\gamma \in (0, 1]$ is an exponential-moving-average rate, ϵ is the numerical constant of Eq. 18, and $\kappa \in (0, 1)$ is a dimensionless tolerance. The scaling in Eq. 9c has a direct interpretation: for two independent latents z_a, z_b drawn from the batch distribution, $\mathbb{E}\|z_a - z_b\|_2^2 = 2 \text{tr}(\text{Cov}(Z))$, so that $\sqrt{2d} \bar{\gamma}_n$ approximates (up to ϵ) the root-mean-square distance between independently drawn states, and κ expresses the completion tolerance as a fraction of this distance. Moreover, whenever the variance hinge is inactive, every per-dimension term satisfies $\text{Var}(Z_{:,j}) + \epsilon \geq \gamma^2$, so the instantaneous estimate in Eq. 9b is at least γ and the threshold is bounded below by $\kappa \sqrt{2d} \gamma$; the VICReg margin thus anchors the scale of τ . Whenever the transition rule is evaluated during training (e.g., to segment demonstrations into milestone phases), $\tau = \tau_n$ is used. Once E_ψ is frozen (Section 4.3), $\bar{\gamma}$ is fixed to its final value $\bar{\gamma}^*$, and deployment uses the constant threshold $\tau = \tau^* = \kappa \sqrt{2d} \bar{\gamma}^*$. When the test in Eq. 9a is satisfied for $m_t = M$, the task is declared complete and the episode terminates.

This neuro-symbolic formulation yields two structural properties. First, the selector output is restricted to the finite set $\mathcal{U}$, which precludes the generation of non-executable or out-of-vocabulary commands. Second, the threshold test turns sub-task completion into an explicit, checkable predicate on the latent state. This predicate does not by itself guarantee that the physical preconditions of the next primitive are satisfied, which depends on the regularity of E_ψ (Section 5.3), nor does it guarantee progress: if a milestone becomes unreachable, the pointer remains at m_t indefinitely. A per-milestone time-out that triggers primitive re-selection can be used to detect such stalls; its design is deferred to the evaluation phase (Section 6).

4.3 Latent Predictor (P_ω)

To generate kinematically stable and physically grounded continuous actions, the control system anticipates the consequences of its decisions. While generative world models achieve this by predicting future pixel observations $\hat{I}_{t+1}$, pixel-level reconstruction can be computationally demanding for high-frequency control loops and may allocate model capacity toward stochastic visual nuisances (e.g., dynamic shadows or moving backgrounds) rather than relevant physical dynamics.

To address this, the VPA framework introduces a predictor P_ω , parameterized as a multi-layer perceptron or causal transformer, that operates within the low-dimensional latent space $\mathcal{Z}$. Given the current latent state z_t and the selected active primitive u_t , the predictor forecasts both the expected future latent state $\hat{z}_{t+1}$ (i.e., ν control steps ahead; Section 3.3) and an associated predictive uncertainty score σ_{t+1} :

$$(\hat{z}_{t+1}, \sigma_{t+1}) = P_\omega(z_t, u_t) \quad (10)$$

The uncertainty score is intended to reflect epistemic uncertainty of the latent dynamics model and is used to modulate the downstream execution horizon. Because the prediction loss below does not supervise σ_{t+1} , we obtain it from the disagreement of a small ensemble [32]: P_ω comprises N_e prediction heads $\{P_\omega^{(i)}\}_{i=1}^{N_e}$ with independent initializations, $\hat{z}_{t+1}$ is the ensemble mean of the head outputs $\hat{z}_{t+1}^{(i)}$, and

$$\sigma_{t+1} = \frac{1}{\sqrt{d} \bar{\gamma}^*} \left(\frac{1}{N_e} \sum_{i=1}^{N_e} \|\hat{z}_{t+1}^{(i)} - \hat{z}_{t+1}\|_2^2 \right)^{1/2},$$

where normalization by the frozen latent scale $\bar{\gamma}^*$ (Eq. 9b) renders σ_{t+1} dimensionless, so that the sensitivity hyperparameter in Eq. 15 does not depend on the scale of $\mathcal{Z}$.

A known challenge in joint-embedding architectures lacking a pixel-level decoder is the vulnerability to representation collapse, in which the encoder and predictor output a constant vector to minimize prediction error. To mitigate this, P_ω and the encoder E_ψ are jointly optimized using a non-collapsing prediction objective based on Variance-Invariance-Covariance Regularization (VICReg) [19]. The objective consists of a latent ℓ_2 prediction loss, which plays the role of the invariance term, coupled with a distributional regularizer:

$$\mathcal{L}_{\text{JEPA}} = \mathbb{E}_t \left[\|\hat{z}_{t+1} - \text{sg}(E_{\bar{\psi}}(I_{t+1}))\|_2^2 \right] + \mathcal{L}_{\text{anti-collapse}}(Z), \quad \mathcal{L}_{\text{anti-collapse}}(Z) = \lambda_v v(Z) + \lambda_c c(Z) \quad (11)$$

where $z_{t+1} = E_{\bar{\psi}}(I_{t+1})$ is the target embedding produced by a momentum (exponential-moving-average) copy $E_{\bar{\psi}}$ of the encoder, $\text{sg}(\cdot)$ denotes the stop-gradient operator, and, with an ensemble, the prediction term is applied to each head $\hat{z}_{t+1}^{(i)}$. The regularizer is evaluated on the batch Z of *online*-encoder latents $z_t = E_\psi(I_t)$, since the momentum encoder receives no gradients; the variance and covariance terms $v(Z)$ and $c(Z)$ are defined in Section 5.2. These terms encourage the variance of each latent dimension to remain above the margin γ and reduce the off-diagonal covariance to decorrelate the latent variables.

This regularized formulation encourages the prediction error $\|\hat{z}_{t+1} - z_{t+1}\|_2^2$ to capture task-relevant physical state evolutions (e.g., object displacement, gripper state changes) associated with the primitive u_t . Consequently, $\hat{z}_{t+1}$ provides the downstream continuous action solver with predictive physical context, substituting a forward-simulated latent state for the autoregressive reasoning of large language models.

Training proceeds in stages. In the representation-learning stage, E_ψ and P_ω are trained jointly with Eq. 11, with P_ω conditioned on the discrete primitive labels u_t provided by the demonstration data. At the end of this stage, both E_ψ and P_ω are frozen, which fixes the latent space, its scale $\bar{\gamma}^*$, and the deployment threshold τ^* . Freezing provides the downstream solver with a stationary conditioning distribution rather than latent inputs that drift as the representation evolves, which simplifies the learning problem for the motor control module; it does not by itself guarantee the semantic consistency of $\mathcal{Z}$, which depends on the quality of the learned representation.

4.4 Continuous Action Chunking via Flow-Matching (π_θ^l)

While the higher-level modules (Sections 4.2 and 4.3) operate in the semantic and latent predictive spaces, the low-level solver π_θ^l grounds these representations into high-frequency motor commands. To manage compounding execution errors and account for sensorimotor latency, we formulate π_θ^l as an action chunking policy.

At each decision step t , rather than predicting a single atomic action, the solver synthesizes an H -step continuous action trajectory chunk $A_t = (a_t, a_{t+1}, \dots, a_{t+H-1}) \in \mathbb{R}^{H \times d_a}$, where d_a denotes the dimensionality of the robot’s control space (e.g., 7-DoF joint velocities and 1-DoF gripper state) and $H \equiv H_{\max}$ is the fixed length of the generated chunk.

To bridge the latent dynamics with physical embodiment and task intent, we construct a unified conditioning vector e_t that encapsulates intent, current state, predicted future, and real-time kinematics:

$$e_t = \text{Concat}(\text{Embed}(u_t), z_t, \hat{z}_{t+1}, S_t, c_{\text{text}}) \in \mathbb{R}^{d_e} \quad (12)$$

where $\text{Embed} : \mathcal{U} \rightarrow \mathbb{R}^{d_u}$ is a learned embedding of the discrete primitive, $z_t, \hat{z}_{t+1} \in \mathbb{R}^d$ are the current and predicted latents, $S_t \in \mathbb{R}^{d_s}$ is the instantaneous proprioceptive state of the robot, $c_{\text{text}} \in \mathbb{R}^{d_c}$ is the text embedding, and $d_e = d_u + 2d + d_s + d_c$. The continuous components enter Eq. 12 after the fixed standardization described in Section 4.5.

Generating multimodal trajectory distributions $p(A_t | e_t)$ often relies on Denoising Diffusion Probabilistic Models (DDPMs) [33, 11]. However, the iterative sampling procedure of DDPMs, which typically requires many denoising steps, can conflict with the ≥ 30 Hz latency target of our architecture.

We therefore parameterize π_θ^l using a Conditional Flow-Matching (CFM) vector field $v_\theta : \mathbb{R}^{H \times d_a} \times [0, 1] \times \mathbb{R}^{d_e} \rightarrow \mathbb{R}^{H \times d_a}$, which transports a simple base distribution (a standard Gaussian) to the action-chunk distribution via an Ordinary Differential Equation (ODE) [12]. Let $\rho \in [0, 1]$ denote the continuous integration time of the flow, distinct from the discrete control step t and from the latent completion threshold τ . We define a linear interpolation path between Gaussian noise $\xi \sim \mathcal{N}(0, I)$, $\xi \in \mathbb{R}^{H \times d_a}$, and the ground-truth action chunk A_t :

$$A_t^{(\rho)} = \rho A_t + (1 - \rho) \xi \quad (13)$$

The neural vector field $v_\theta(A_t^{(\rho)}, \rho | e_t)$ is trained to regress the time-derivative of this path, i.e., the target velocity $A_t - \xi$, using the Flow-Matching objective:

$$\mathcal{L}_{\text{solver}} = \mathbb{E}_{t, \xi, \rho \sim \mathcal{U}[0, 1]} \left[\|v_\theta(A_t^{(\rho)}, \rho | e_t) - (A_t - \xi)\|_2^2 \right] \quad (14)$$

To mitigate the risk of physical errors caused by inaccurate latent forecasting, the system employs an uncertainty-aware execution strategy. Rather than always executing the full generated chunk, the executed horizon is scaled based on the predictor’s uncertainty. We define the executed horizon H_t at decision step t as:

$$H_t = \max(H_{\min}, \lfloor H_{\max} \cdot \exp(-\beta \bar{\sigma}_{t+1}) \rfloor) \quad (15)$$

where $\beta > 0$ is a sensitivity hyperparameter, $H_{\min} \leq H_{\max} = H$ bound the executed chunk length, and $\bar{\sigma}_{t+1}$ is a smoothed uncertainty signal. Using the raw score σ_{t+1} directly would allow high-frequency fluctuations in the uncertainty estimate to toggle the controller between long and short horizons on consecutive decision steps, which can induce jitter at the motor level. We therefore apply an asymmetric (fast-attack, slow-release) filter, $\bar{\sigma}_{t+1} = \max\{\sigma_{t+1}, (1 - \alpha_\sigma)\bar{\sigma}_t + \alpha_\sigma \sigma_{t+1}\}$ with $\alpha_\sigma \in (0, 1]$, under which the horizon contracts immediately when uncertainty rises but expands only gradually as it decays. When the environment dynamics are predictable (low $\bar{\sigma}_{t+1}$), a long prefix of the chunk is executed; during high-uncertainty events, H_t contracts toward $H_{\min}$, so that the control loop relies more frequently on fresh sensory feedback I_t rather than on the latent prediction.

During closed-loop execution, a chunk $\hat{A}_t \in \mathbb{R}^{H \times d_a}$ is sampled by drawing $\xi \sim \mathcal{N}(0, I)$ and integrating the learned vector field from $\rho = 0$ to $\rho = 1$ with K steps of a numerical ODE solver; with the Euler method, $A^{(\rho_{k+1})} = A^{(\rho_k)} + K^{-1} v_\theta(A^{(\rho_k)}, \rho_k | e_t)$ for $\rho_k = k/K$, $k = 0, \dots, K-1$, and $A^{(0)} = \xi$. The first H_t actions of $\hat{A}_t$ are then executed. Although the conditional paths in Eq. 13 are straight, the learned marginal flow can be curved, particularly for strongly multimodal action distributions; few-step integration is therefore accurate only to the extent that the learned flow is close to straight, a property that rectification procedures explicitly promote [30]. In practice, a small number of steps $K \in \{1, 2, 3\}$ is commonly used. Each decision step thus requires K sequential evaluations of v_θ , each of which processes the full $H \times d_a$ chunk in parallel: the number of sequential evaluations is independent of H , whereas the arithmetic cost of each evaluation grows with $H \cdot d_a$ (Section 5.1). This bounded sequential depth is intended to allow the hardware to execute the current chunk open-loop at 30–100 Hz while the next chunk is being computed.

4.5 Representation Alignment and Solver Warm-Starting

Before deploying end-to-end continuous action chunking, we adopt an initial alignment phase that matches the representation used by the frozen JEPA modules to the input expected by the continuous Flow-Matching Solver π_θ^l .

Because the latent space is learned independently of the action data, its coordinates may have arbitrary offsets and a scale $\bar{\gamma}^*$ that differs substantially from that of the proprioceptive and text inputs. Conditioning the vector field on unnormalized latents can produce large gradients in the early iterations of Eq. 14. After freezing, we therefore standardize the latent components of e_t with fixed statistics, $\tilde{z} = (z - \bar{\mu}_Z)/\bar{\gamma}^*$, applied to both z_t and $\hat{z}_{t+1}$, where $\bar{\mu}_Z$

is the mean latent over the training data; the proprioceptive state S_t is standardized analogously using dataset statistics. Because the same scale $\bar{\gamma}^*$ also determines τ^* and normalizes σ_{t+1} , the completion threshold, the uncertainty signal, and the solver inputs are expressed in mutually consistent units.

Utilizing principles of Behavior Cloning, the solver is then warm-started on demonstration datasets: conditioning vectors e_t are computed by the frozen encoder and predictor along demonstration trajectories and paired with the corresponding action chunks, and v_θ is trained with Eq. 14. This alignment phase is intended to prevent the low-level solver from misinterpreting the latent transitions before any subsequent fine-tuning of the vector field. By establishing a shared representation early in training, the action decoder is aligned with the stationary latent space of the frozen modules, which is expected to facilitate a smoother and more reliable optimization of the flow-matching objective.

5 Theoretical Analysis and System Properties

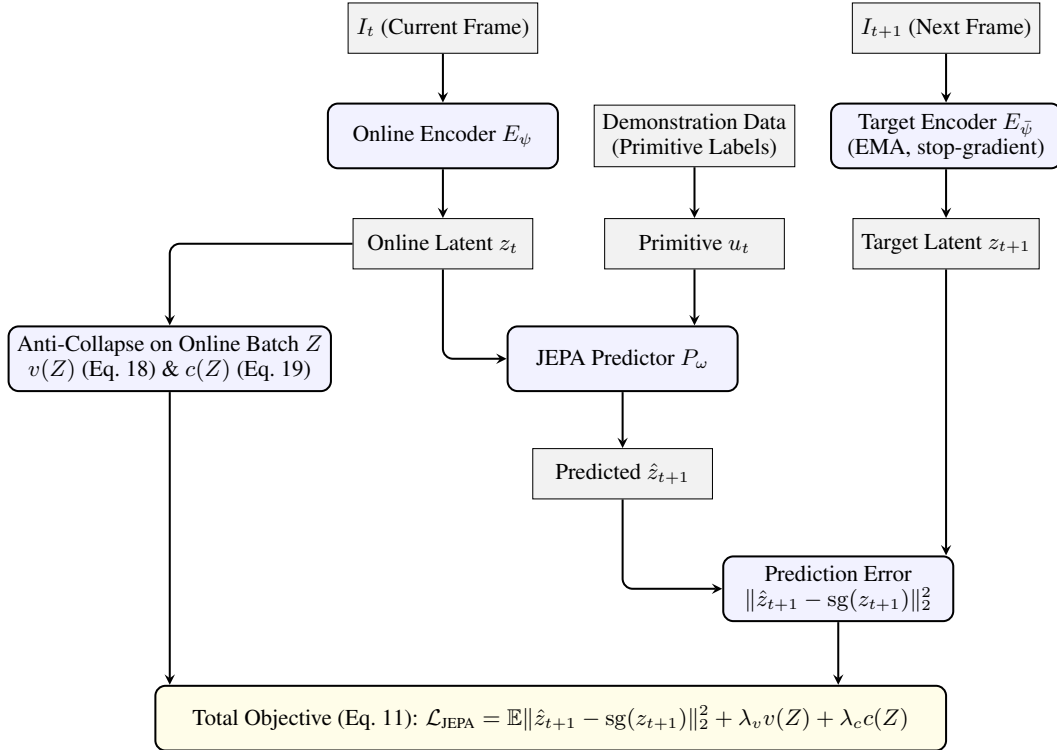


Figure 3: Offline Representation-Learning Objective. The online encoder E_ψ and the predictor P_ω are trained jointly. The target latent z_{t+1} is produced by a momentum (exponential-moving-average) copy $E_{\bar{\psi}}$ of the encoder under stop-gradient, and the VICReg variance and covariance terms are evaluated on the batch Z of online latents, since the target branch receives no gradients. During this stage, the predictor is conditioned on the discrete primitive labels u_t supplied by logged demonstration trajectories (Eq. 10); the frozen text encoder is not involved. At the end of the stage, E_ψ and P_ω are frozen, which fixes the latent scale $\bar{\gamma}^*$ and the deployment threshold τ^* (Eq. 9).

This section analyzes the runtime complexity of the VPA pipeline, the properties of its regularized latent representation, and the conditions under which its discrete sub-goal mechanism limits error accumulation across task phases. We distinguish between statements that follow directly from the definitions of the architecture and its objectives (Propositions 5.1 and 5.2) and statements that additionally rely on regularity properties of the learned encoder (Proposition 5.3). Because the latter properties cannot be enforced by the training objective or by architectural constraints alone, the corresponding conclusions are presented as conditional results and as empirical expectations to be verified in the evaluation of Section 6.

5.1 Computational Complexity and Latency

A primary limitation of contemporary VLA models is their reliance on autoregressive token generation for continuous control. Let H be the length of the required action chunk and d_a the action dimension. Token-based VLAs discretize

the continuous action space and generate $N = q \cdot H \cdot d_a$ tokens per inference cycle, where q is the number of tokens per scalar action component.

For an LLM-based policy with parameters W_{LLM} , generating N tokens requires N forward passes, each of which depends on the token produced by its predecessor and therefore cannot be executed in parallel. With key-value caching, the i -th pass costs $\Theta(|W_{\text{LLM}}| + L_i d_{\text{model}})$ arithmetic operations, where L_i is the attention context length at step i and d_{model} the hidden width. Moreover, at batch size one, each pass must read all decoder parameters from memory; for models whose parameters exceed on-chip memory, the wall-clock latency therefore satisfies

$$T_{\text{VLA}} = \sum_{i=1}^N t_{\text{pass}}(L_i) \geq \frac{N \cdot |W_{\text{LLM}}|}{\text{BW}}, \quad N = q \cdot H \cdot d_a, \quad (16)$$

where $t_{\text{pass}}(L_i)$ is the wall-clock time of the i -th pass and BW denotes the effective memory bandwidth, measured in parameters per unit time. The latency of autoregressive action decoding thus grows at least linearly in N , and hence in H . Reported closed-loop frequencies for such systems lie at approximately 2–5 Hz ($T_{\text{VLA}} \approx 200\text{--}500$ ms) [4], which restricts reactive, high-speed physical adjustments.

To analyze the VPA pipeline, we distinguish two cost measures for a single decision step: the *sequential depth* D , defined as the length of the longest chain of network evaluations in which each evaluation requires the output of its predecessor, and the total arithmetic *work* W . Wall-clock latency is lower-bounded by the sequential depth times the minimum per-evaluation latency, and approaches the work divided by the available parallelism when the hardware is saturated.

Proposition 5.1 (Sequential depth and work of a VPA decision step). *Consider one decision step of VPA in which the solver generates a chunk of length H with K integration steps, and assume that the milestone embeddings $z_g^{(m)}$ and the text embedding c_{text} have been computed once per episode. Then:*

- (i) *the sequential depth is $D_{\text{VPA}} = K + 3$, which is independent of H and d_a , whereas autoregressive decoding of $N = q \cdot H \cdot d_a$ tokens has sequential depth $D_{\text{VLA}} = N = \Theta(H \cdot d_a)$;*
- (ii) *the work is $W_{\text{VPA}}(H) = C_E + C_h + C_P + K \cdot C_l(H)$, where C_E , C_h , and C_P are the costs of the encoder, the selector, and the predictor (including all N_e ensemble heads), none of which depends on H , and $C_l(H)$ is the cost of one evaluation of v_θ . Since v_θ outputs an element of $\mathbb{R}^{H \times d_a}$, $C_l(H) = \Omega(H \cdot d_a)$; if v_θ applies self-attention across the H chunk positions, $C_l(H)$ additionally contains a term quadratic in H .*

Proof. (i) Within one decision step, the data dependencies form the chain $I_t \rightarrow z_t \rightarrow u_t \rightarrow (\hat{z}_{t+1}, \sigma_{t+1}) \rightarrow e_t \rightarrow A^{(\rho_1)} \rightarrow \dots \rightarrow A^{(\rho_K)}$: the selector requires z_t (Eq. 8), the predictor requires u_t (Eq. 10), and each Euler step requires the previous iterate (Section 4.4). This chain contains one evaluation each of E_ψ , π_ϕ^h , and P_ω , and K evaluations of v_θ ; the ensemble heads of P_ω are mutually independent and may be evaluated in parallel. No evaluation in the chain is repeated as a function of H . For autoregressive decoding, token i is an input to pass $i + 1$, so all N passes lie on a single dependency chain. (ii) The expression for W_{VPA} follows by summing the costs along the chain. Each evaluation of v_θ writes $H \cdot d_a$ output entries, which requires $\Omega(H \cdot d_a)$ operations. $\square$

Proposition 5.1 makes precise the sense in which the VPA solver removes the dependence on the action horizon. The wall-clock latency of a decision step is

$$T_{\text{VPA}} = \underbrace{t_E + t_h + t_P}_{\text{independent of } H} + K \cdot t_l(H), \quad (17)$$

where t_E , t_h , t_P , and $t_l(H)$ denote the wall-clock times of the encoder, the selector, the predictor, and one solver evaluation, respectively. Because $K \ll N$ and the sequential dependency on output generation is removed, the number of sequential network evaluations drops from $\Theta(H \cdot d_a)$ to $K + 3 = \mathcal{O}(K)$. This does *not* imply that the per-cycle cost is $\mathcal{O}(1)$ in H : by part (ii) of Proposition 5.1, the arithmetic work of each solver evaluation grows at least linearly with $H \cdot d_a$, and $t_l(H)$ is approximately constant in H only while a single evaluation of v_θ fits within the available hardware parallelism; beyond that regime, $t_l(H)$ grows with $C_l(H)$. The reduction in sequential depth is shared with other non-autoregressive action heads (Section 2.4); the distinguishing latency property of VPA is that all networks on the dependency chain are intended to be lightweight relative to a vision-language backbone. The target end-to-end latency of ≤ 10 ms and the target control frequency of 30–100 Hz depend on the sizes of E_ψ , π_ϕ^h , P_ω , and v_θ and on the deployment hardware; they are design objectives to be validated empirically (Section 6), not consequences of Proposition 5.1.

5.2 Representation Regularization and Anti-Collapse Properties

In predictive architectures trained with a mean-squared-error objective and no pixel-level decoder, the encoder and predictor admit a degenerate solution: representation collapse. The vision encoder E_ψ and predictor P_ω can drive the prediction term $\mathbb{E}[\|\hat{z}_{t+1} - z_{t+1}\|_2^2]$ to zero by mapping all observations to an identical constant vector $\zeta \in \mathbb{R}^d$.

To exclude this degenerate solution, VPA incorporates the variance and covariance terms of VICReg [19]. Given a batch of B online-encoder latents $Z \in \mathbb{R}^{B \times d}$ evaluated during training, the anti-collapse loss acts through two penalties.

First, to counteract collapse to a point mass, a hinge loss is applied to the standard deviation of each latent dimension $j \in \{1, \dots, d\}$ with target margin $\gamma > 0$:

$$v(Z) = \frac{1}{d} \sum_{j=1}^d \max\left(0, \gamma - \sqrt{\text{Var}(Z_{:,j})} + \epsilon\right) \quad (18)$$

where $\epsilon > 0$ is a small numerical constant, with $\epsilon < \gamma^2$, that prevents unbounded gradients at zero variance, and Var is computed with the same estimator as $C(Z)$ below. The term $v(Z)$ vanishes if and only if $\text{Var}(Z_{:,j}) \geq \gamma^2 - \epsilon$ for every j . It is a penalty rather than a hard constraint: at an optimum of the joint objective at which $v(Z) > 0$, some dimensions may remain below the margin. Furthermore, since $v(Z)$ is a batch statistic, it does not imply that E_ψ is injective on the state space; physically distinct states may still be mapped to nearby or identical latents. The hinge also does not bound the per-dimension standard deviations from above, which is why the completion threshold is calibrated to the realized margin $\bar{\gamma}$ rather than to γ alone (Eq. 9c).

Second, to counteract dimensional collapse, in which several latent dimensions encode the same variable, we penalize the off-diagonal entries of the empirical covariance matrix $C(Z) \in \mathbb{R}^{d \times d}$:

$$c(Z) = \frac{1}{d} \sum_{i \neq j} ([C(Z)]_{i,j})^2 \quad (19)$$

where $C(Z) = \frac{1}{B-1} \sum_{b=1}^B (Z_{b,:} - \bar{Z})(Z_{b,:} - \bar{Z})^\top$ and $\bar{Z}$ is the batch mean. Equation 19 penalizes linear correlations between latent dimensions; it does not penalize nonlinear dependencies, so decorrelation should not be equated with statistical independence.

The regularization term is $\mathcal{L}_{\text{anti-collapse}}(Z) = \lambda_v v(Z) + \lambda_c c(Z)$ (Eq. 11). The following proposition characterizes what this regularizer does and does not exclude. For tractability, we analyze the idealized objective in which the momentum target encoder coincides with the online encoder, $E_{\bar{\psi}} = E_\psi$, as at a fixed point of the moving-average update, and write $\mathcal{L}(\psi, \omega) = \mathcal{L}_{\text{pred}}(\psi, \omega) + \lambda_v v(Z) + \lambda_c c(Z)$, where $\mathcal{L}_{\text{pred}}$ denotes the prediction term of Eq. 11.

Proposition 5.2 (Anti-collapse properties). *Assume $\lambda_v > 0$ and $\epsilon < \gamma^2$.*

- (i) *Every constant encoder $E_\psi \equiv \zeta$, paired with an optimal predictor, attains the objective value $\mathcal{L}_{\text{const}} = \lambda_v(\gamma - \sqrt{\epsilon})$. Consequently, if the model class contains parameters (ψ', ω') with $\mathcal{L}(\psi', \omega') < \lambda_v(\gamma - \sqrt{\epsilon})$, no constant encoder is a global minimizer.*
- (ii) *If a batch Z satisfies both $v(Z) = 0$ and $c(Z) = 0$, then $C(Z)$ is diagonal with all diagonal entries at least $\gamma^2 - \epsilon > 0$; hence $C(Z)$ is positive definite and the batch latents span all d dimensions of Z . This can occur only if $B > d$.*

Proof. (i) For a constant encoder, every column of Z is constant, so $\text{Var}(Z_{:,j}) = 0$ for all j and $C(Z) = 0$. Each hinge term therefore equals $\max(0, \gamma - \sqrt{\epsilon}) = \gamma - \sqrt{\epsilon}$, giving $v(Z) = \gamma - \sqrt{\epsilon}$, while $c(Z) = 0$. A predictor that outputs ζ attains $\mathcal{L}_{\text{pred}} = 0$, which is optimal since $\mathcal{L}_{\text{pred}} \geq 0$. Hence $\mathcal{L}_{\text{const}} = \lambda_v(\gamma - \sqrt{\epsilon})$, and any parameters with a strictly smaller objective value exclude constant encoders from the set of global minimizers. (ii) $c(Z) = 0$ forces all off-diagonal entries of $C(Z)$ to vanish, and $v(Z) = 0$ forces $[C(Z)]_{j,j} = \text{Var}(Z_{:,j}) \geq \gamma^2 - \epsilon$ for all j . A diagonal matrix with positive diagonal entries is positive definite and has rank d ; since $\text{rank } C(Z) \leq B - 1$, this requires $B > d$. $\square$

Part (i) of Proposition 5.2 refines a frequently stated but imprecise claim: the positivity of λ_v alone does not exclude collapse. Collapse is excluded from the global minimizers only if some non-degenerate representation achieves an objective value below the collapse level $\lambda_v(\gamma - \sqrt{\epsilon})$, a condition that is more readily satisfied for larger λ_v . Part (ii) shows that, at points where both penalties vanish, total and dimensional collapse are excluded on the training batch. Neither part implies that latent distances faithfully reflect physical distances: a non-collapsed representation may still be insensitive to task-relevant differences or sensitive to nuisance factors. Non-collapse is therefore necessary but not sufficient for $\|z_t - z_g^{(m)}\|_2$ to serve as a proxy for task progress; the additional conditions are examined in Section 5.3.

5.3 Error Accumulation and Conditional Phase-Transition Bounds

In step-by-step imitation learning (Behavioral Cloning), policies are subject to compounding covariate shift: if the learned policy errs with probability ϵ_π per step under the demonstrator’s state distribution, its expected total cost over a horizon of T steps (with per-step costs in $[0, 1]$) exceeds that of the demonstrator by at most $\mathcal{O}(T^2\epsilon_\pi)$, and this quadratic dependence is tight in the worst case [34]. VPA addresses error accumulation through two mechanisms: temporal action chunking within each task phase, and discrete sub-goal thresholding between phases.

Action chunking. The Flow-Matching solver π_θ^l generates an H -step action chunk A_t , and executing it open-loop reduces the number of decisions over a horizon of T steps from T to T/H . Applying the same reduction at the chunk level, treating each chunk as a single decision whose cost is the sum of H per-step costs and whose error probability under the demonstrator’s distribution is ϵ_H , yields

$$J(\hat{\pi}) - J(\pi^*) \leq \left(\frac{T}{H}\right)^2 \cdot H \cdot \epsilon_H = \frac{T^2}{H} \epsilon_H,$$

where $J(\hat{\pi})$ and $J(\pi^*)$ are the expected total costs of the learned and demonstrator policies. This bound improves on $T^2\epsilon_\pi$ only if ϵ_H grows sublinearly in H . Since a chunk contains H actions, a union bound yields only $\epsilon_H \leq H\epsilon_\pi$, under which the chunk-level bound coincides with the step-level bound. The benefit of chunking is therefore an empirical property, consistent with prior findings on action chunking [10], rather than a guarantee. Open-loop execution moreover forgoes sensory feedback within each chunk, a trade-off that the adaptive horizon of Eq. 15 is intended to manage.

Sub-goal thresholding. The milestone pointer m_t advances only when $\|z_t - z_g^{(m_t)}\|_2 < \tau$ (Eq. 9a). Whether this latent condition constrains the physical state depends on the regularity of the composite map from task-relevant physical state to latent, $y \mapsto E_\psi(\mathcal{I}(y, \eta))$ (Section 3.1). We state two conditions on this map, with a slack $\delta_\eta \geq 0$ that accounts for the latent displacement caused by variation of the nuisance factors η within the deployment set $\mathcal{N}$; without loss of generality, the same slack is used in both.

Assumption 5.1 (Upper regularity). There exist $L_E < \infty$ and $\delta_\eta \geq 0$ such that, for all $y, y' \in \mathcal{Y}$ and all $\eta, \eta' \in \mathcal{N}$,

$$\|E_\psi(\mathcal{I}(y, \eta)) - E_\psi(\mathcal{I}(y', \eta'))\|_2 \leq L_E \|y - y'\|_2 + \delta_\eta.$$

Assumption 5.2 (Lower (co-)Lipschitz) regularity). There exist $\mu_E > 0$ and $\delta_\eta \geq 0$ such that, for all $y, y' \in \mathcal{Y}$ and all $\eta, \eta' \in \mathcal{N}$,

$$\|E_\psi(\mathcal{I}(y, \eta)) - E_\psi(\mathcal{I}(y', \eta'))\|_2 \geq \mu_E \|y - y'\|_2 - \delta_\eta.$$

Proposition 5.3 (Conditional phase-transition bounds). Let $y_g^{(m)}$ denote the target physical configuration of milestone m , observed as $I_g^{(m)} = \mathcal{I}(y_g^{(m)}, \eta_g^{(m)})$ with $\eta_g^{(m)} \in \mathcal{N}$, and let $z_g^{(m)} = E_\psi(I_g^{(m)})$.

(i) Soundness. Under Assumption 5.2, a satisfied threshold test implies physical proximity to the milestone:

$$\|z_t - z_g^{(m_t)}\|_2 < \tau \implies \|y_t - y_g^{(m_t)}\|_2 < \frac{\tau + \delta_\eta}{\mu_E}. \quad (20)$$

(ii) Completeness. Under Assumption 5.1 and $\tau > \delta_\eta$, sufficient physical proximity triggers the transition:

$$\|y_t - y_g^{(m_t)}\|_2 < \frac{\tau - \delta_\eta}{L_E} \implies \|z_t - z_g^{(m_t)}\|_2 < \tau.$$

(iii) Phase-onset reset. Under Assumption 5.2, let t_k be the decision step at which the pointer advances from k to $k + 1$, and let $r_{k+1} = \|y_{t_k} - y_g^{(k)}\|_2$ denote the physical residual with respect to milestone k at the onset of phase $k + 1$. Then $r_{k+1} < (\tau + \delta_\eta)/\mu_E$, irrespective of the trajectory executed during phases $1, \dots, k$.

Proof. (i) Applying Assumption 5.2 with $(y, \eta) = (y_t, \eta_t)$ and $(y', \eta') = (y_g^{(m_t)}, \eta_g^{(m_t)})$ gives $\mu_E \|y_t - y_g^{(m_t)}\|_2 - \delta_\eta \leq \|z_t - z_g^{(m_t)}\|_2 < \tau$; rearranging yields Eq. 20. (ii) Applying Assumption 5.1 to the same pair gives $\|z_t - z_g^{(m_t)}\|_2 \leq L_E \|y_t - y_g^{(m_t)}\|_2 + \delta_\eta < (\tau - \delta_\eta) + \delta_\eta = \tau$. (iii) At t_k , the threshold test is satisfied for milestone k by the definition of Eq. 9a; part (i) then gives the bound, which depends only on τ , δ_η , and μ_E . $\square$

Remark 5.1 (Direction of the Lipschitz inequality). An upper bound of the form $\|z_a - z_b\|_2 \leq L_E \|y_a - y_b\|_2$ (Assumption 5.1 with $\delta_\eta = 0$) implies only $\|y_t - y_g^{(m_t)}\|_2 \geq \|z_t - z_g^{(m_t)}\|_2 / L_E$, which is a *lower* bound on the physical deviation. An upper Lipschitz condition alone therefore cannot bound the physical error at a transition from above; that conclusion requires the lower regularity of Assumption 5.2. The two assumptions play complementary roles: Assumption 5.2 makes the threshold test sound (a transition implies physical proximity), whereas Assumption 5.1 makes it complete (physical proximity triggers a transition).

Consequences and their limits. Under Assumption 5.2, part (iii) of Proposition 5.3 implies $\max_k r_k < (\tau^* + \delta_\eta)/\mu_E$ over all phases that are entered: the physical residual with which each primitive begins is bounded uniformly and independently of the length M of the milestone sequence, so that deviation accumulated in one phase is not propagated beyond this bound into the next. Three qualifications apply. First, the bound concerns phase onsets only; deviations within a phase are governed by the low-level policy and are subject to the chunk-level analysis above. Second, the bound is conditional on each transition occurring: by part (ii), a transition is guaranteed only if the physical state enters the ball of radius $(\tau^* - \delta_\eta)/L_E$ around the milestone, which requires $\tau^* > \delta_\eta$, that is, a threshold exceeding the latent displacement induced by nuisance variation; otherwise the pointer may stall (Section 4.2). Third, τ^* and δ_η jointly determine a trade-off: increasing κ in Eq. 9c raises τ^* , which improves completeness under nuisance variation but loosens the soundness bound of Eq. 20.

Why these properties are expectations rather than guarantees. The constants L_E , μ_E , and δ_η are properties of the learned encoder composed with image formation, and none of them is controlled by the objective of Eq. 11. Regarding Assumption 5.1, standard dot-product self-attention is not globally Lipschitz [21], so a Vision Transformer encoder possesses no global Lipschitz bound with respect to its input unless explicit constraints are imposed. On a bounded image domain a finite constant exists, but it may be very large and is neither estimated nor controlled during training. Spectral normalization of the linear layers [20], combined with a Lipschitz-continuous attention variant [21], can bound the pixel-space Lipschitz constant L_ψ of E_ψ . Even then, Assumption 5.1 concerns the composition with the image-formation map $\mathcal{I}$, whose local sensitivity to y can be very large at occlusion and contact events, so that L_E is bounded by L_ψ times the Lipschitz constant of $\mathcal{I}$ only where the latter is finite. Spectral normalization is not part of the base formulation; its effect on representation quality and on the threshold mechanism is a subject for empirical evaluation. Regarding Assumption 5.2, a lower bound cannot be obtained from architectural constraints of this kind at all. The encoder is designed to be non-injective with respect to pixels, being intended to be invariant to η , so the lower bound must hold with respect to y alone. It therefore depends on the data and on the learned representation, and it is violated wherever task-relevant differences are not visible in the image, for example under occlusion or object symmetry. The variance term of Eq. 18 promotes only batch-level spread and does not imply Assumption 5.2.

Accordingly, we do not claim that VPA guarantees non-compounding error. We instead state the expectation that, to the extent that the learned encoder satisfies Assumptions 5.1 and 5.2 with favorable constants on the deployment distribution, discrete sub-goal thresholding limits the propagation of physical error across task phases in the manner described by Proposition 5.3. This expectation is directly testable. In simulation, where the physical state y_t is available, the constants μ_E , L_E , and δ_η can be estimated from the lower and upper envelopes of $\|z - z'\|_2$ as a function of $\|y - y'\|_2$ over sampled state pairs and nuisance conditions, and the phase-onset residuals r_k can be measured directly (Section 6).

6 Proposed Evaluation Methodology (Future Work)

Section 5 establishes two kinds of properties: properties that follow from the architecture and its objectives (Propositions 5.1 and 5.2), and properties that hold only under regularity assumptions on the learned encoder (Proposition 5.3). Empirical validation is therefore required for two purposes: to quantify the practical control advantages of the VPA framework, and to test whether the assumptions underlying the conditional analysis hold for a trained encoder. We propose an evaluation methodology designed to benchmark high-frequency reactive control and visual robustness, which are areas in which current autoregressive Vision-Language-Action (VLA) models often experience difficulties, together with targeted measurements of the theoretical quantities introduced in Section 5. No empirical results are reported in this work.

6.1 Simulation Environments

To reproducibly benchmark the system prior to physical hardware deployment, we propose utilizing established GPU-accelerated simulation platforms:

- **NVIDIA Isaac Sim** [35]: This platform is selected for its PhysX physics engine and ray-traced rendering. It is suited to testing robustness against visual distribution shifts, such as dynamic lighting or reflections, and to assessing whether the regularized JEPA representations remain informative without pixel reconstruction.
- **ManiSkill3** [36]: This simulator is selected for its standardized manipulation tasks and parallelized tensor-based API. It allows for the rapid evaluation of continuous action chunking and neuro-symbolic transitions across numerous episodic rollouts.

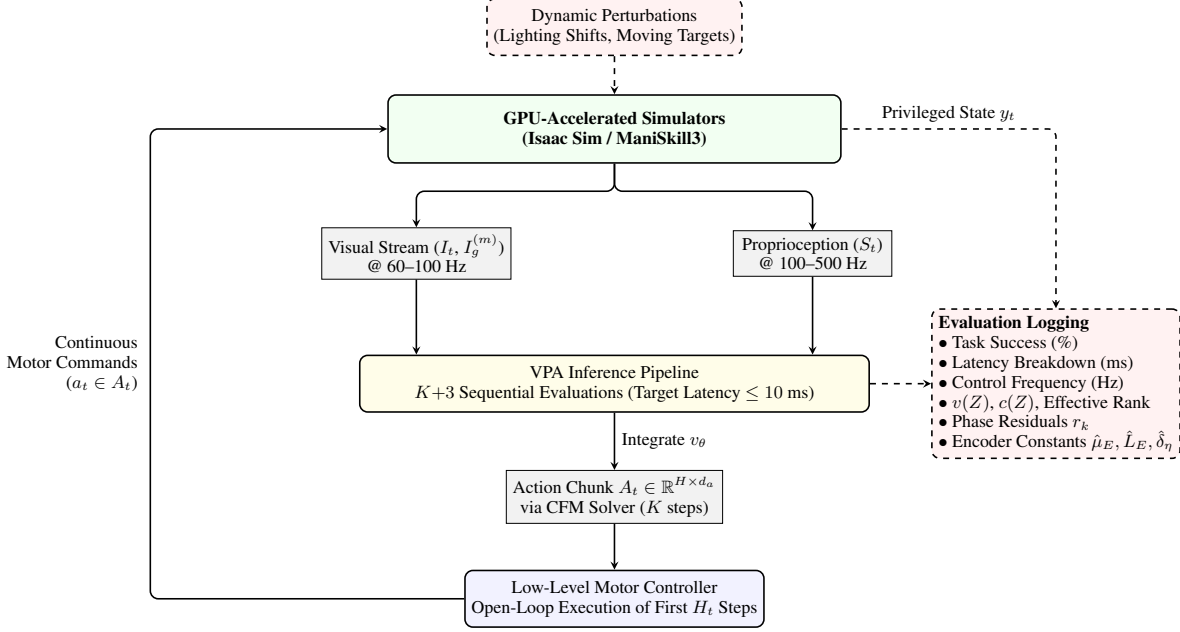


Figure 4: Proposed Implementation and Closed-Loop Evaluation Architecture. To benchmark high-frequency reactive control, the Goal-Conditioned VPA framework is deployed in GPU-accelerated simulators subjected to dynamic physical and visual perturbations. High-frequency sensory streams are processed by the VPA pipeline, whose sequential depth of $K + 3$ network evaluations per decision step is independent of the action horizon (Proposition 5.1), and the resulting action chunks are executed open-loop for an uncertainty-dependent number of steps H_t . This design is intended to avoid the ≤ 5 Hz bottleneck reported for autoregressive VLA architectures. Because the simulator exposes the privileged physical state y_t , the evaluation also logs the quantities required to test the conditional analysis of Section 5.3.

Both platforms expose the ground-truth physical state y_t (robot configuration and object poses), which is unavailable on physical hardware and is required for the measurements of Section 6.3.

6.2 Proposed Benchmark Tasks

The empirical evaluation focuses on dynamic, non-stationary environments in which the targeted 30–100 Hz control frequency of VPA may provide a measurable advantage over the 2–5 Hz frequencies reported for autoregressive VLAs.

1. High-Speed Reactive Pick-and-Place: Unlike static tabletop manipulation, this task involves moving targets, such as items on a moving conveyor belt or objects physically perturbed mid-grasp. Autoregressive models often experience action lag in these scenarios, executing stale trajectories based on outdated observations. The Flow-Matching solver (π_θ^l) will be evaluated on its ability to update action chunks at > 50 Hz to track and grasp moving targets.

2. Reactive Obstacle Avoidance: In this task, the robot is instructed to reach a visual goal, but dynamic obstacles are introduced into the workspace mid-trajectory. This evaluates the neuro-symbolic primitive selector (π_ϕ^h) and the discrete sub-goal thresholding ($\|z_t - z_g^{(m_t)}\|_2 < \tau$). If an obstacle blocks a trajectory, the system is expected to halt milestone progression or switch primitives to mitigate error accumulation; the frequency and duration of such halts also serve to characterize pointer stalls (Section 4.2).

3. Visual Nuisance Generalization: To test the nuisance robustness motivated in Sections 4.1 and 5.2, and to estimate the nuisance slack δ_η of Section 5.3, the agent will be trained in a visually sterile environment and tested in environments with visual distractors, such as changing background textures, moving camera perspectives, and variable illumination. The metric of interest is the zero-shot success rate decay compared to pixel-reconstruction baselines.

4. Language Intent Disambiguation: To evaluate the system’s semantic comprehension, this task presents environments in which visual goals are ambiguous without language context (e.g., distinguishing between an instruction to move a cup and an instruction to throw it away). This benchmark tests whether the text embedding c_{text} produced by the lightweight frozen encoder provides sufficient conditioning for the primitive selector and the flow-matching solver,

which are the modules that receive it (Eqs. 8 and 12), to resolve the overarching task intent without a large language model.

6.3 Verification of Theoretical Properties and Ablations

Beyond task-level performance, we propose the following measurements, each of which targets a specific statement or design choice of Sections 4 and 5:

1. **Latency scaling (Proposition 5.1):** Measure T_{VPA} and its components t_E , t_h , t_P , and $t_l(H)$ (Eq. 17) on stated hardware as H and K are varied, in order to identify the range of H over which $t_l(H)$ is approximately constant and the point at which it begins to grow with the chunk dimensionality.
2. **Encoder regularity (Assumptions 5.1 and 5.2):** Using the privileged state y_t , sample state pairs under controlled nuisance variation and estimate $\hat{\mu}_E$, $\hat{L}_E$, and $\hat{\delta}_\eta$ from the lower and upper envelopes of $\|z - z'\|_2$ as a function of $\|y - y'\|_2$ (e.g., via quantile regression), with and without spectral normalization of the encoder.
3. **Phase-onset residuals (Proposition 5.3):** Record the residual r_k at every milestone transition and compare its empirical distribution with the conditional bound $(\tau^* + \hat{\delta}_\eta)/\hat{\mu}_E$, together with the rates of premature transitions, missed transitions, and stalls under a per-milestone time-out.
4. **Threshold calibration (Eq. 9):** Compare a fixed absolute threshold with the variance-margin-scaled threshold of Eq. 9c across training checkpoints, and sweep the tolerance κ to trace the trade-off between soundness and completeness described in Section 5.3.
5. **Horizon stability (Eq. 15):** Compare the raw uncertainty score σ_{t+1} with the filtered signal $\bar{\sigma}_{t+1}$ in terms of the frequency of horizon switches and the magnitude of action discontinuities at chunk boundaries.

6.4 Baselines and Evaluation Metrics

The VPA architecture will be benchmarked against autoregressive VLA models (e.g., RT-2 [1]), generalist policies with continuous action heads (e.g., Octo [22] and π_0 [14]), and diffusion-based behavior cloning policies [33, 11]. Including continuous-head baselines isolates the contribution of the lightweight latent pipeline from that of non-autoregressive action generation, since the latter reduction in sequential depth is shared by these methods (Section 5.1). The primary evaluation metrics will include:

1. **Task Success Rate (%)**: Measured across static and dynamic environments.
2. **End-to-End Inference Latency (ms)**: Wall-clock time from image capture to motor command generation, reported together with its per-module breakdown and the deployment hardware.
3. **Control Frequency (Hz)**: The maximum stable closed-loop execution rate.
4. **Representation Health**: The variance and covariance penalties $v(Z)$ and $c(Z)$, together with the effective rank of $C(Z)$ [37], evaluated for the frozen visual encoder under visual shift.
5. **Phase-Transition Fidelity**: The distribution of phase-onset residuals r_k and the rates of premature transitions, missed transitions, and stalls.
6. **Execution Smoothness**: The frequency of changes in the executed horizon H_t and the magnitude of action discontinuities at chunk boundaries.

7 Conclusion

The pursuit of generalized embodied intelligence has recently been supported by the scaling of autoregressive Vision-Language-Action (VLA) models. However, the sequential token generation on which many of these models rely makes them challenging to deploy for the high-frequency, reactive demands of physical robotic control.

In this paper, we introduced the Goal-Conditioned Vision-Predictor-Action (VPA) framework, proposing a shift from text-conditioned autoregressive decoding to latent-predictive continuous control. By decoupling high-level task logic from low-level motor execution and utilizing a lightweight frozen text encoder to capture user instructions, the VPA framework aims to retain semantic task comprehension without relying on computationally heavy Large Language Models (LLMs).

Our analysis establishes three results at different levels of strength. First, formulating multi-step action chunking through Conditional Flow-Matching (CFM) reduces the number of sequential network evaluations per decision step

from $\Theta(H \cdot d_a)$ for autoregressive token decoding to $K + 3$, a quantity independent of the action horizon H ; the arithmetic cost of each solver evaluation nevertheless still scales with the chunk dimensionality $H \cdot d_a$, so the per-cycle cost is not $\mathcal{O}(1)$ in H . Second, the VICReg variance and covariance penalties exclude fully collapsed representations from the global minimizers of the joint-embedding objective whenever a non-degenerate representation attains a lower objective value, and they exclude dimensional collapse on batches where both penalties vanish; the realized variance margin further provides a principled scale for calibrating the milestone completion threshold τ . Third, discrete sub-goal thresholding bounds the physical residual at each phase transition by $(\tau + \delta_\eta)/\mu_E$, but only under a co-Lipschitz condition on the encoder with respect to task-relevant physical state. Because Vision Transformers are not globally Lipschitz without explicit constraints, and because this lower bound cannot be enforced through architectural constraints alone, we regard the containment of error across task phases as an empirical expectation rather than a formal guarantee.

Limitations. The repertoire of high-level behaviors is bounded by the fixed primitive dictionary $\mathcal{U}$, and zero-shot synthesis of skills outside $\mathcal{U}$ is not addressed by the present formulation. The target inference latency of ≤ 10 ms and the target control frequency of 30–100 Hz are design objectives whose attainment depends on the sizes of the constituent networks and on the deployment hardware. No empirical results are reported in this work; the evaluation protocol of Section 6 is designed to measure task-level performance as well as the quantities on which our conditional analysis rests, namely the encoder constants μ_E , L_E , and δ_η and the phase-onset residuals r_k .

We therefore view VPA as an analytically characterized architectural proposal whose theoretical properties are stated together with the assumptions they require. As we move toward empirical validation in dynamic simulation environments, we expect this explicit separation between unconditional and conditional properties to provide a sound basis for developing fast and physically grounded robotic manipulation systems.

References

- [1] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- [2] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [3] Siyuan Huang, Zhengkai Jiang, Hao Dong, Yu Qiao, Peng Gao, and Hongsheng Li. Instruct2act: Mapping multi-modality instructions to robotic actions with large language model. *arXiv preprint arXiv:2305.11176*, 2023.
- [4] Ziyue Zhu, Shangyang Wu, Shuai Zhao, Zhiqiu Zhao, Shengjie Li, Yi Wang, Fang Li, and Haoran Luo. Ns-vla: Towards neuro-symbolic vision-language-action models. *arXiv preprint arXiv:2603.09542*, 2026.
- [5] Jingwen Sun, Wenyao Zhang, Zekun Qi, Shaojie Ren, Zezhi Liu, Hanxin Zhu, Guangzhong Sun, Xin Jin, and Zhibo Chen. Vla-jepa: Enhancing vision-language-action model with latent world model. *arXiv preprint arXiv:2602.10098*, 2026.
- [6] Yann LeCun et al. A path towards autonomous machine intelligence version 0.9. 2, 2022-06-27. *Open Review*, 62(1):1–62, 2022.
- [7] Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15619–15629. IEEE, 2023.
- [8] Lucas Maes, Quentin Le Lidec, Damien Scieur, Yann LeCun, and Randall Balestriero. Leworldmodel: Stable end-to-end joint-embedding predictive architecture from pixels. *arXiv preprint arXiv:2603.19312*, 2026.
- [9] Wancong Zhang, Basile Terver, Artem Zhohus, Soham Chitnis, Harsh Sutaria, Mido Assran, Randall Balestriero, Amir Bar, Adrien Bardes, Yann LeCun, et al. Hierarchical planning with latent world models. *arXiv preprint arXiv:2604.03208*, 2026.
- [10] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- [11] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10-11):1684–1704, 2025.
- [12] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

- [13] Michael S Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. *arXiv preprint arXiv:2209.15571*, 2022.
- [14] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [15] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020.
- [16] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.
- [17] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021.
- [18] Hadas Kress-Gazit, Georgios E Fainekos, and George J Pappas. Temporal-logic-based reactive mission and motion planning. *IEEE transactions on robotics*, 25(6):1370–1381, 2009.
- [19] Adrien Bardes, Jean Ponce, and Yann LeCun. Vicreg: Variance-invariance-covariance regularization for self-supervised learning. *arXiv preprint arXiv:2105.04906*, 2021.
- [20] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. *arXiv preprint arXiv:1802.05957*, 2018.
- [21] Hyunjik Kim, George Papamakarios, and Andriy Mnih. The lipschitz constant of self-attention. In *International Conference on Machine Learning*, pages 5562–5571. PMLR, 2021.
- [22] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [23] Yongchao Huang and Hassan Raza. Knowledge, rules and their embeddings: Two paths towards neuro-symbolic jepa. *arXiv preprint arXiv:2603.13265*, 2026.
- [24] Yongchao Huang. Sjepa: Learning elegant latent dynamics with hybrid symbolic-neural predictors. *arXiv preprint arXiv:2608.04060*, 2026.
- [25] Yeonseok Lee. NS-JEPA: Towards Efficient Embodied AI via Neuro-Symbolic Joint-Embedding Predictive Architecture. Zenodo, <https://doi.org/10.5281/zenodo.22493915>, 2026.
- [26] Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics. *arXiv preprint arXiv:2511.08544*, 2025.
- [27] Mahmoud Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, Artem Zhohus, Sergio Arnaud, Abha Gejji, Ada Martin, Francois Robert Hogan, Daniel Dugas, Piotr Bojanowski, Vasil Khalidov, Patrick Labatut, Francisco Massa, Marc Szafraniec, Kapil Krishnakumar, Yong Li, Xiaodong Ma, Sarath Chandar, Franziska Meier, Yann LeCun, Michael Rabbat, and Nicolas Ballas. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.
- [28] Shangchen Miao, Ningya Feng, Jialong Wu, Ye Lin, Xu He, Dong Li, and Mingsheng Long. Jepa-vla: Video predictive embedding is needed for vla models. *arXiv preprint arXiv:2602.11832*, 2026.
- [29] Aleksandar Vujinović and Aleksandar Kovačević. Act-jepa: Novel joint-embedding predictive architecture for efficient policy representation learning. *IEEE Access*, 2026.
- [30] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.
- [31] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.
- [32] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in neural information processing systems*, 30, 2017.
- [33] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

- [34] Stéphane Ross and Drew Bagnell. Efficient reductions for imitation learning. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 661–668. JMLR Workshop and Conference Proceedings, 2010.
- [35] NVIDIA. Isaac Sim. <https://developer.nvidia.com/isaac/sim>, 2026. Accessed: 2026-10-08.
- [36] Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse-kai Chan, et al. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *arXiv preprint arXiv:2410.00425*, 2024.
- [37] Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *2007 15th European signal processing conference*, pages 606–610. IEEE, 2007.